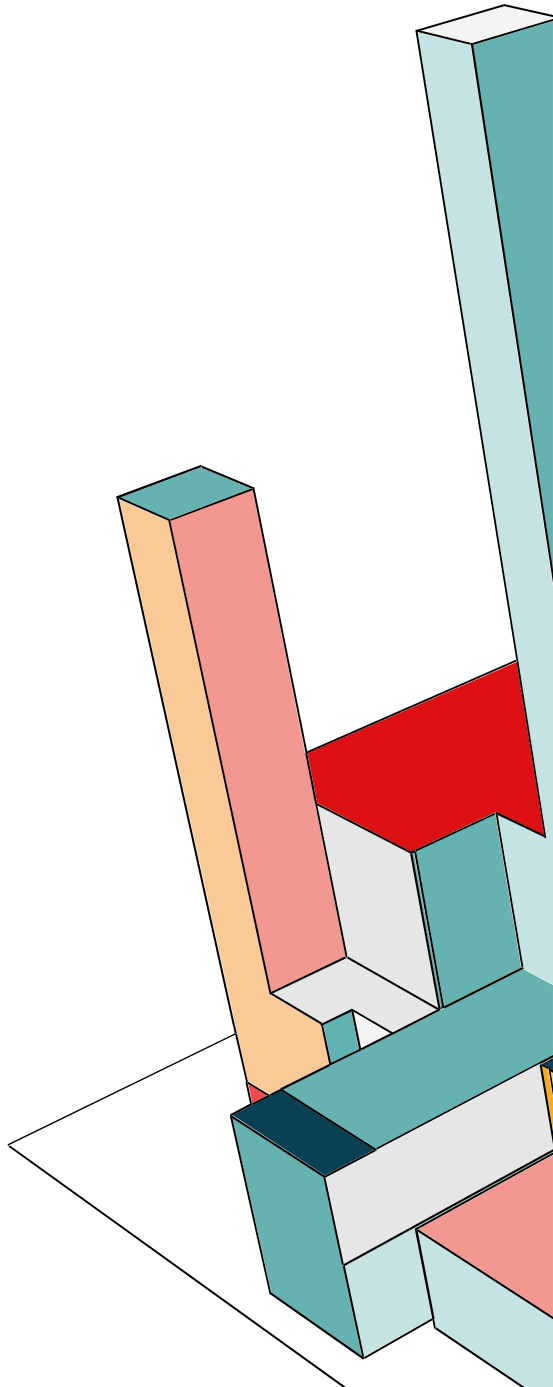


КОНСТРУИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

ПЛАН ЛЕКЦИИ 14

1. DevOps — подход к разработке программного обеспечения
2. DevSecOps — культура безопасной разработки
3. Observability — принципы и техники наблюдения за системой



Введение

Исторический контекст:

DevOps возник как ответ на “противостояние” между разработчиками и системными администраторами.

Актуальность сегодня:

Современный IT — это облака, микросервисы и гонка за Time-to-Market.



Кто такой Девопс?

Мастер на все руки для решения всех проблем бизнеса с универсальным стеком?

или

специалист-виртуоз конкретной области, в совершенстве знающий ограниченный набор инструментов?



С чем можно столкнуться, к чему надо быть готовым

1. Компания не понимает кто это и зачем он нужен
2. Представляет, кто это и зачем нужен, но они нанимают спасателя, или переименовывают своих и ждут магию
3. В компаниях разный стек, разные требования



За что отвечает, за что — нет

- Автоматизация процесса сборки и деплоя
- Формирование flow pipeline
- Развертывание тестовых сред
- Участие в решении проблем dev в части сборки и деплоя
- Участие в решении проблем ops в части деплоя
- Могут участвовать в подготовке, построении и обслуживании прод инфры
- Могут участвовать в вопросах управления артефактами
- Могут разрабатывать и внедрять процессы управления изменениями и управления конфигурацией

- Заправляет картриджи для принтеров
- Прокладывает витую пару по офису
- Продает продукты вместе с бизнесом
- Поддерживает прод
- Пишут код бизнес логики
- Занимаются нагрузочным тестированием
- Работают над архитектурой приложения
- Оптимизируют работу СУБД

Типовые этапы разработки цифрового продукта

- проектирование
- разработка архитектуры
- этап планирования проекта и сроков и его разработки
- старт разработки
- приемо-сдаточные испытания
- вывод в продакшен

Выделим проблемы такого подхода

- недостаточность проработки на каждом этапе
- неизбежный разрыв между ожиданием и реальностью
- **срыв** сроков
- **затягивание** сроков проекта
- напряженность в коммуникации между всеми участниками жизненного цикла продукта

и многое другое

Что беспокоило разработку?

- консервативные взгляды на новые технологии
- непрекращающиеся **дедлайны**
- количество и качество менеджмента
- непонимание и «палки в колёса» со стороны безопасников
- **недоговороспособность** других участников
- непредсказуемый релизный цикл
- непонимание технологий и практик, используемых другими участниками

Что беспокоило администраторов?

- изменчивый стек технологий разработки
- необходимость поддержки различных ОС
- **сложность поддержки** инфраструктуры «снежинки»
- **невозможность воспроизведения** окружения разработки в инфраструктуре тестирования и продакшена
- недостаточное понимание специфики используемых языков разработки
- непредсказуемый релизный цикл

Что беспокоило информационную безопасность?

- **отсутствие** понимания, как устроена работа всех участников
- отсутствие поддержки со стороны бизнеса
- **невыполнение** выданных замечаний
- отсутствие влияния на процесс разработки
- второстепенный статус у других участников
- отсутствие рычагов давления и контроля за процессом разработки и эксплуатации
- перевод чрезмерной ответственности на ИБ
- непредсказуемый релизный цикла

Что беспокоило бизнес?

- **непрогнозируемые** релизы
- срыв сроков
- **увеличение стоимости** проекта
- отсутствие здорового контакта внутри ИТ (разработка, тестирование, администраторы, информационная безопасность)

Как же на самом деле выглядел процесс?

- проектирование
- проработка архитектуры



- осознание разницы между запланированным и сделанным
- поиск **виноватых**
- повторение тех же **ошибок**

Почему всё это неэффективно?

- отсутствие коммуникации ухудшает ситуацию с каждым новым спринтом разработки
- непонимание приводит к негативу
- негатив приводит к неразумным компромиссам
- ошибки накапливаются, достижения сгорают
- стоимость и сроки исправления ошибок увеличивается

«Чем раньше обнаружить и исправить ошибку, тем дешевле это обойдется»

Енг Кек Нгуен, «Тестирование программного обеспечения. Фундаментальные концепции менеджмента бизнес-приложений»

Короче, страдали все

- бизнес хочет побыстрее зафиксировать какую-то прибыль
- разработка хочет писать код и не спорить с другими участниками
- администраторы хотят понятных требований к инфраструктуре и воспроизводимость окружения
- безопасность хочет быть услышанной и востребованной

Три главных проблемы

Проблема #1

отсутствие своевременного **взаимодействия** между всеми участниками жизненного цикла цифрового продукта

Три главных проблемы

Проблема #2

непрекращающееся развитие и **усложнение** технологий

- Прогресс увеличивает производство
- Производство увеличивает количество рабочих инструментов
- Инструменты воскрешают старые и порождают новые ошибки и уязвимости

Три главных проблемы

Проблема #3

перекладывание **ответственности**
и импульсивность в принятии важных решений

Мы подвластны внезапным импульсам и всегда предпочитаем настоящее будущему

Современный взгляд на разработку цифрового продукта

- изменение методологии
- своевременное и качественное **взаимодействие** всех участников
- стандартизация процессов
- воспроизводимость окружений
- **баланс** между удобством, скоростью и безопасностью

Возникновение devops

devops – методология активного взаимодействия специалистов по разработке со специалистами по информационно-технологическому обслуживанию и взаимная **интеграция их рабочих процессов** для обеспечения высокого качества продукта

devops ⟶ devsecops

- процессы ci/cd всё еще подвержены задержкам и критике со стороны участников процесса, особенно ИБ
- стоимость исправления ошибок и закрытия уязвимостей **все еще высока**
- качественное взаимодействие с безопасностью все еще **отсутствует**

Кто такой инженер devops? Чем он занимается, какие инструменты использует и сколько зарабатывает?

Кто же такой инженер devops?

Инженер devops понимает необходимость в гибкой методологии разработки продукта:

- осознает **ценность** практик devops
- вникает в существующие потребности и **проблемы** всех участников
- владеет локальным окружением, способен визуализировать рабочие процессы
- имеет опыт работы с облачной инфраструктурой
- работает с конфигурацией и инфраструктурой в декларативном ключе

Инфраструктура как код

Инженер devops работает с инфраструктурой и конфигурацией в декларативном ключе:

- управление инфраструктурой
- стандартизация образов
- инструменты **декларативного** управления инфраструктурой, конфигурацией и сервисами (**terraform**, **ansible**)

Непрерывная сборка и доставка

Инженер devops управляет конвейером непрерывной сборки и доставки продукта (ci/cd):

- технологии контейнеризации (**docker**, **podman**)
- микросервисная архитектура
- обеспечение непрерывной сборки и доставки продукта (**ci/cd**)
- обеспечение безопасности контейнеров и микросервисов

Мониторинг, логирование, аудит, трассировка

Инженер devops использует в работе системы мониторинга, логирования, трассировки и аудита информационных систем и инфраструктуры:

- модели и принципы работы систем **мониторинга**
- инструменты **логирования** и **аудита**
- сбор и обработка логов приложений и инфраструктуры
- поддержка трассировки приложений по взаимодействию с командами разработки

Как же выглядит инструментарий инженера devops?

Система контроля версий:

- Gitlab (bitbucket, github, mercurial)

ci/cd:

- Gitlab CI (jenkins, teamcity)

Контейнеризация:

- docker (podman, containerd, cri-o, runc)

Оркестрация:

- kubernetes (swarm, nomad, mesos)

Управление конфигурацией:

- ansible (chef, puppet)

Управление инфраструктурой:

- terraform (cloudformation)

Мониторинг, логирование, трейсинг, визуализация:

- prometheus, jaeger, ELK, loki, grafana, kiali, etc..

Безопасность:

- trivy, clair, falco, gatekeeper, kube-audit, kube-hunter, etc...

DevSecOps-анализ:

- SAST, SCA, DAST, IAST, RASP

Эволюция DevOps: что меняется?

DevOps не стоит на месте. Рассмотрим ключевые тренды:

- GitOps
- DevSecOps
- MLOps
- Микросервисы и Serverless
- Мультиклауд

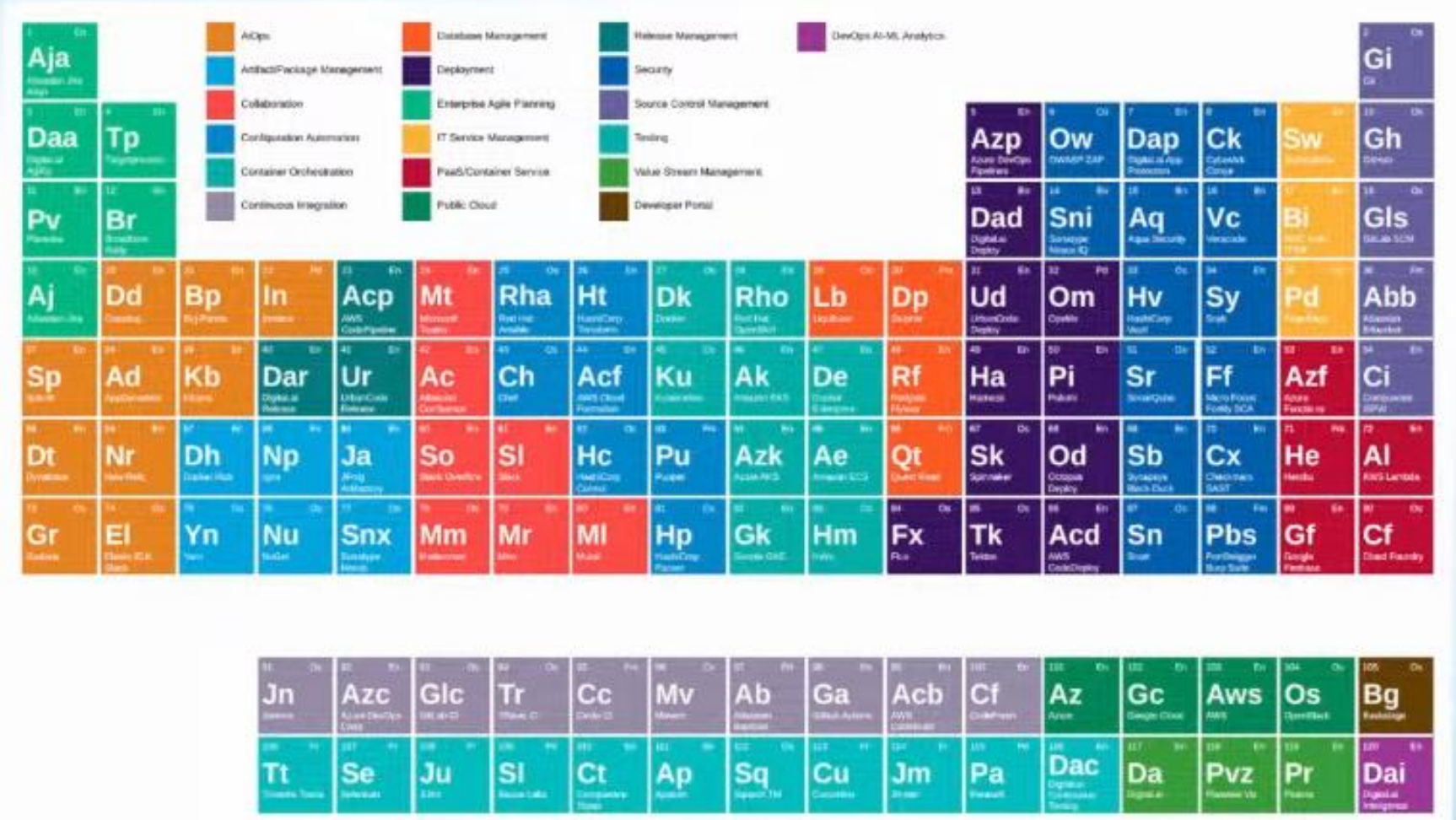


Актуальные инструменты DevOps-инженера

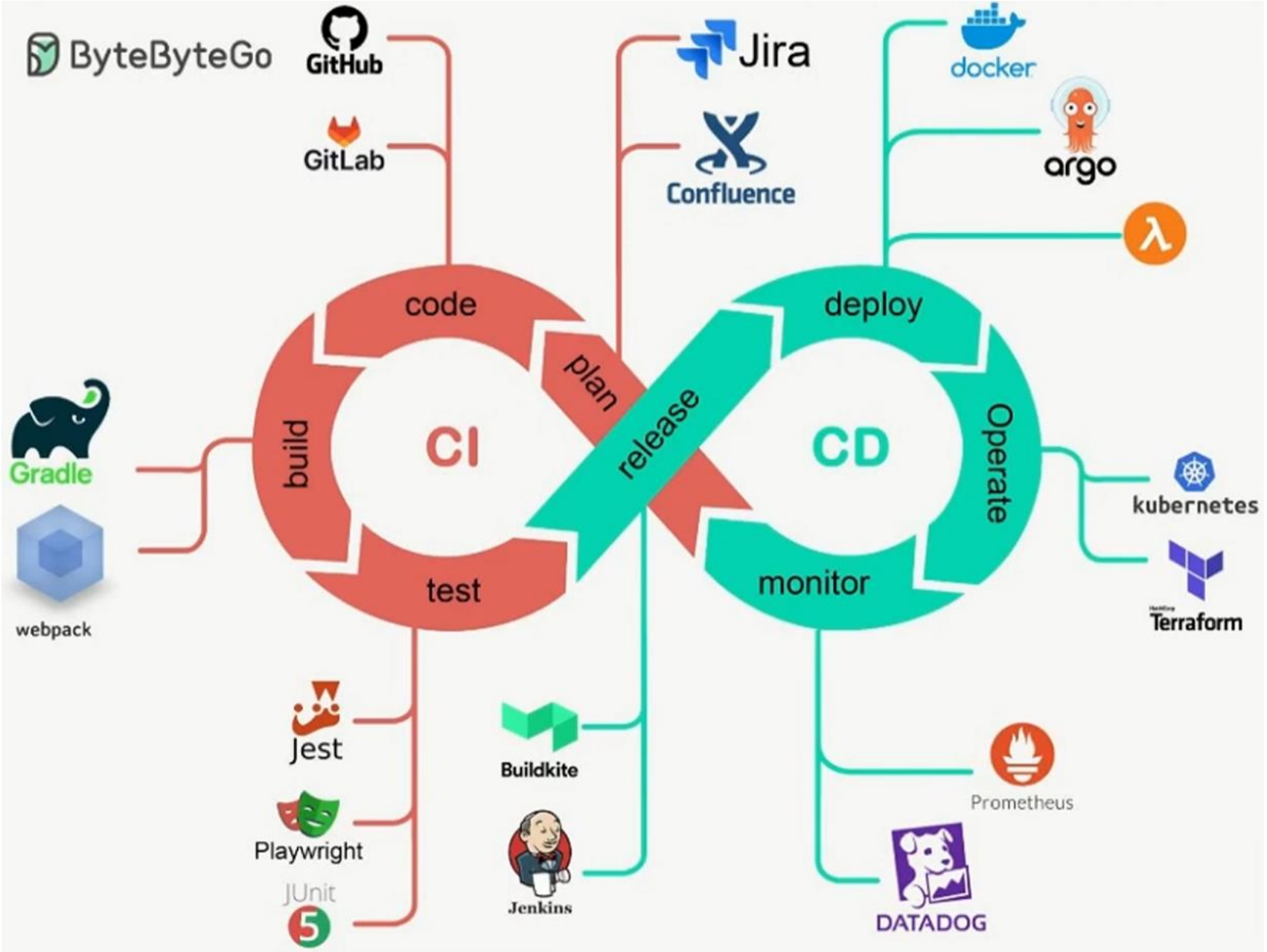
- **CI/CD (Continuous Integration/Continuous Delivery)**
  - Jenkins, GitLab CI/CD, GitHub Actions, CircleCI, ArgoCD
  - Особенности: интеграция с облачными провайдерами, поддержка контейнеров.
- **Инфраструктура как код (IaC)**
  - Terraform, AWS CloudFormation.
  - Ansible, Chef, Puppet для конфигурации.
  - OpenTofu (альтернатива Terraform), Crossplane для универсального управления облаками.
- **Контейнеризация и оркестрация**
  - Docker, Podman.
  - Kubernetes (и его экосистема: Helm, Kustomize).
- **Мониторинг и логирование**
  - Prometheus + Grafana, Datadog, New Relic.
  - ELK Stack (Elasticsearch, Logstash, Kibana), Loki.
- **Безопасность**
  - HashiCorp Vault для управления секретами.
  - Clair для сканирования образов контейнеров.



Периодическая система инструментов devops



scan me!


ByteByteGo
GitHub
GitLab
Jira
Confluence
docker
argo
λ
code
plan
build
CI
test
release
deploy
Operate
CD
monitor
Gradle
webpack
kubernetes
Terraform
Jest
Playwright
JUnit
5
Buildkite
Jenkins
Prometheus
DATADOG

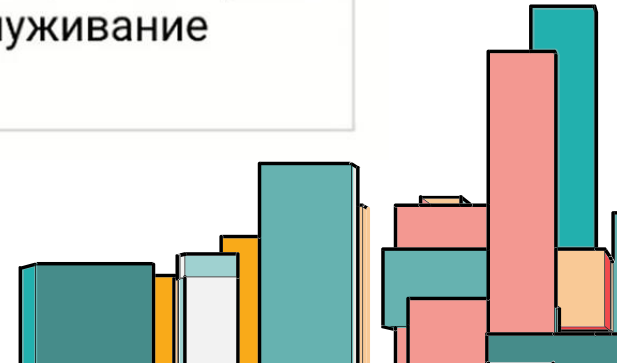
DevOps vs...

- **DevOps vs. SRE (Site Reliability Engineering)**
  - SRE: фокус на надежности, SLA/SLO, автоматизации инцидентов.
  - DevOps: культура сотрудничества, end-to-end автоматизация процессов.
  - Пример: SRE внедряет Error Budgets, DevOps обеспечивает CI/CD-пайплайны.
- **DevOps vs. Platform Engineer**
  - Platform Engineer: создание внутренних платформ для разработчиков (например, Backstage).
  - DevOps: обеспечение инструментов и процессов для этих платформ.
- **DevOps vs. Cloud Engineer**
  - Cloud Engineer: углубленная работа с облачными сервисами (AWS/Azure/GCP).
  - DevOps: интеграция облака в CI/CD и инфраструктуру.
- **Где пересекаются роли?**
  - Общие инструменты (Kubernetes, Terraform).
  - Общая цель: снижение операционных затрат и повышение скорости разработки.



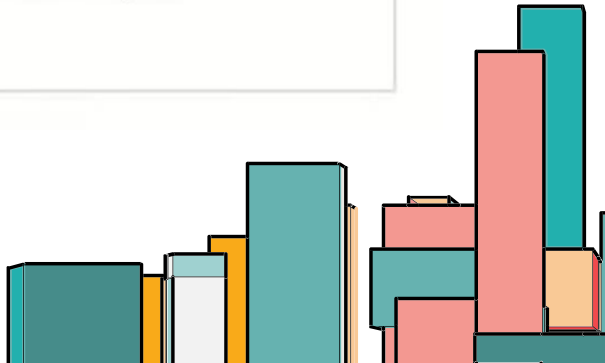
Цели и мера успешности

Методология	Основная цель	Метрики успеха	Подход
DevOps	Сотрудничество между командами разработки и эксплуатации	Частота развёртывания, время выполнения задач, MTTR	Непрерывная интеграция, непрерывная доставка, непрерывное развертывание
SRE	Управление надежностью	Доступность, надежность, масштабируемость	Бюджет ошибок, цели уровня сервиса (SLO), индикаторы уровня сервиса (SLI)
Platform Engineer	Создание и управление платформами (продукт)	Время выхода на рынок, масштабируемость платформы	Автоматизация, устойчивость, мониторинг и самообслуживание



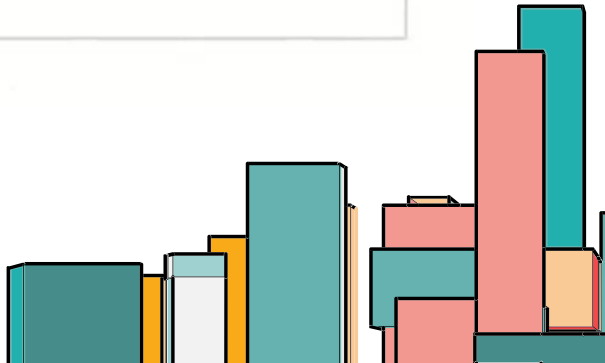
Ценности, культура, команда

Методология	Подход к культуре	Подход к ценностям	Подход к структуре команды
DevOps	Ориентация на сотрудничество	Непрерывное улучшение, экспериментирование	Команды с кросс-функциональными ролями
SRE	Ориентация на надежность	Learning from failure	SRE-команды, отдельные SRE инженеры
Platform Engineer	Ориентация на продукт	Гибкость, масштабируемость	Самоорганизация



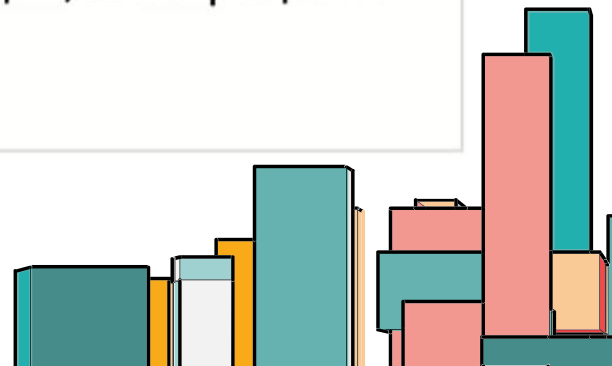
Инфраструктура и конфигурации

Методология	Подход к управлению инфраструктурой	Подход к управлению конфигурациями
DevOps	Инфраструктура как код (IaC), автоматическое развертывание	Конфигурация как код
SRE	Индикаторы уровня сервиса (SLI), цели уровня сервиса (SLO)	Версионирование конфигураций
Platform Engineer	Автоматизация инфраструктуры, оркестрация ресурсов	Управление конфигурациями



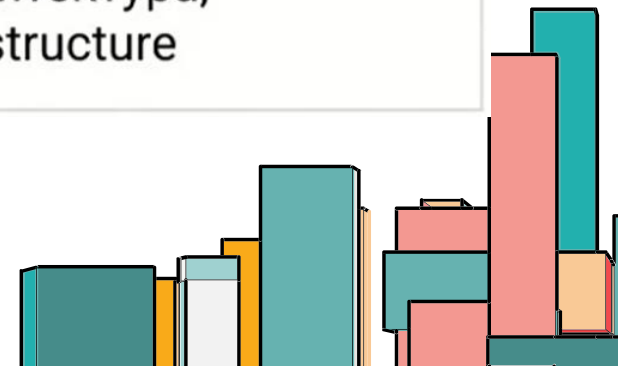
Дорожная карта

DevOps	SRE	Platform Engineer
1. Культурные переход к сотрудничеству и экспериментам	1. Внедрение ролей и обязанностей SRE	1. Принятие платформенного подхода
2. Внедрение инструментов и практик автоматизации	2. Разработка индикаторов и целей уровней сервиса (SLI/SLO)	2. Разработка стандартного тулсета и процессов
3. Реализация процесса непрерывной интеграции и доставки	3. Внедрение blameless анализа для непрерывного улучшения	3. Автоматизация, интеграция и доставка



Управление рисками

Методология	Подход к оценке рисков	Подход к принятию рисков
DevOps	Shift Left (DevSecOps), инфраструктура как код	Культура обучения и экспериментирования, Chaos Engineering
SRE	Бюджет ошибок, планирование мощности	Принятие отказов, Blameless postmortem
Platform Engineer	Аудит безопасности, Threat Modeling	Устойчивая архитектура, Immutable Infrastructure



Ключевые задачи DevOps

- **Автоматизация всего**
  - От тестирования до развертывания и масштабирования.
- **Безопасность как часть пайплайна**
  - Shift-left security: раннее обнаружение уязвимостей.
- **Управление сложностью распределенных систем**
  - Решение проблем observability (метрики, логи, трейсы).
- **Поддержка AI/ML проектов**
  - Интеграция MLOps в существующие процессы.



Почему DevOps останется?

- Это мост между людьми, процессами и технологиями.
- Даже с появлением AI-инструментов, человеческая экспертиза незаменима.

Советы:

- Учите принципы, а не только инструменты (например, The Twelve-Factor App).
- Развивайте «soft skills»: DevOps — это про коммуникацию.



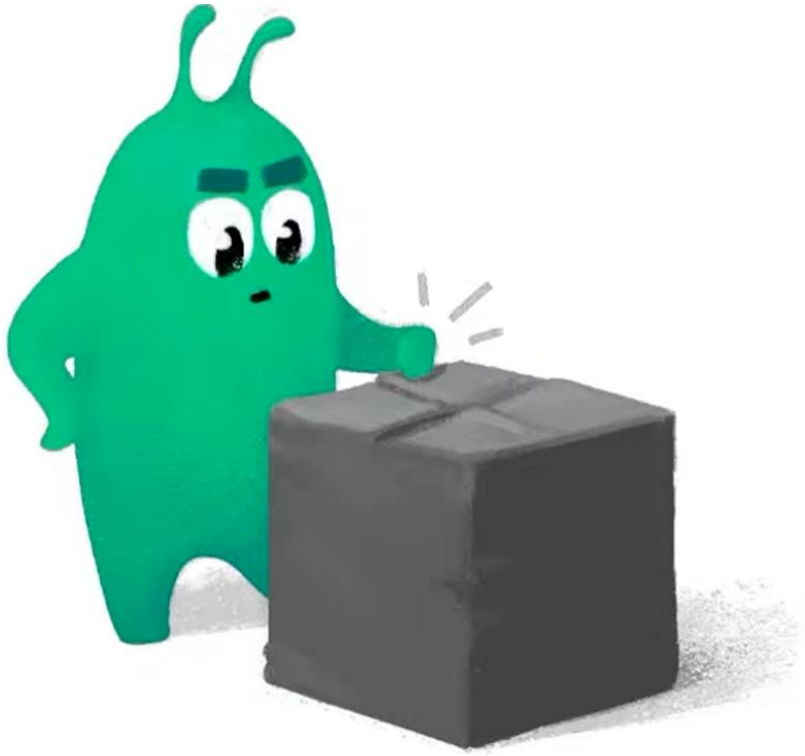
Мы переименовали **админов**
в DevOps инженеров,
и вышло неплохо!



Теперь нужно «отдевопсить» мониторинг, переименовав его в Observability



ТИПЫ МОНИТОРИНГА



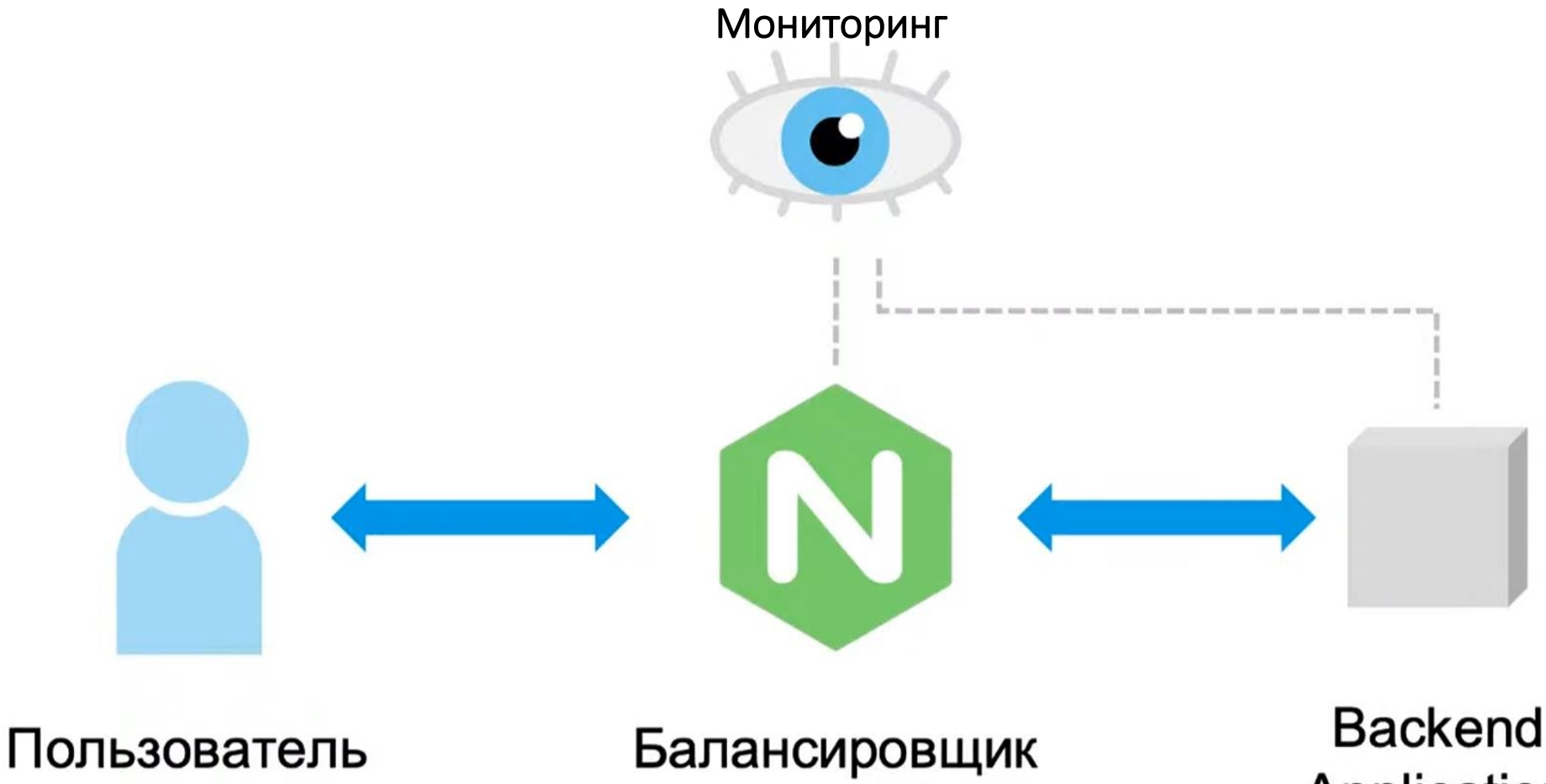
Blackbox мониторинг



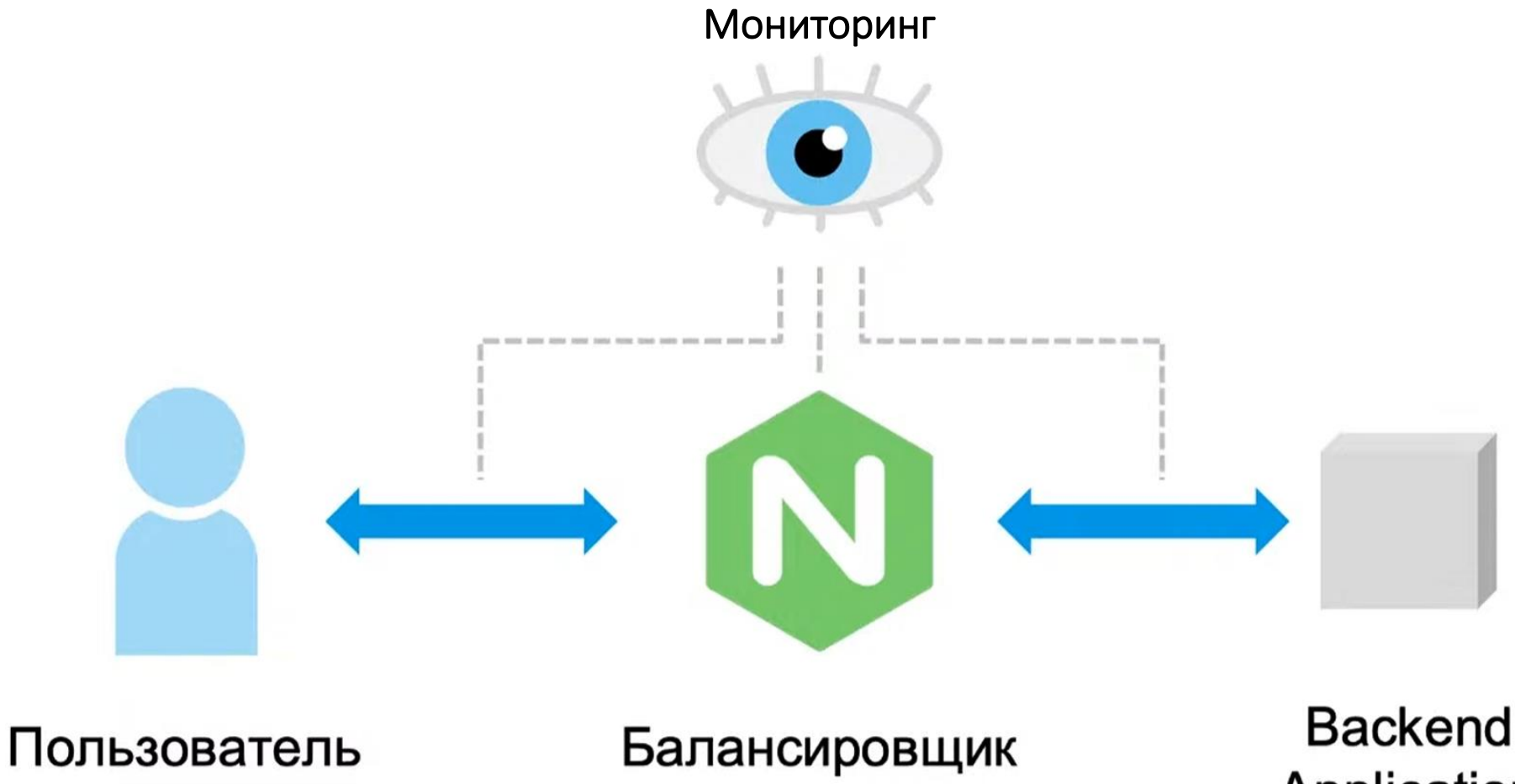
Whitebox мониторинг



BLACKBOX МОНИТОРИНГ



WHITEBOX МОНИТОРИНГ



Разница между мониторингом и Observability

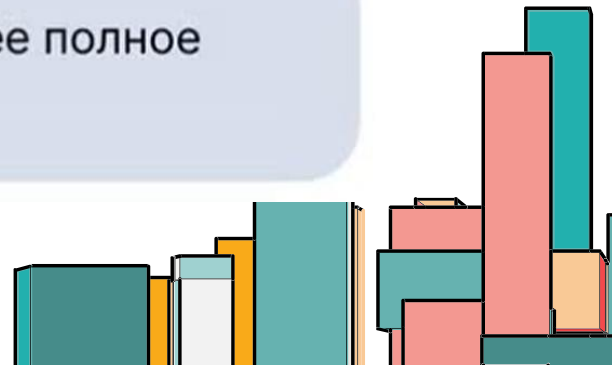
Мониторинг:

- Сбор и анализ данных о состоянии системы
- Фокус на метриках и алертах
- Помогает обнаруживать проблемы и реагировать на них

Observability:

- Глубокое понимание внутреннего состояния системы
- Включает логи, метрики и трассировки
- Позволяет анализировать причины проблем и оптимизировать систему

Итог: Мониторинг — это часть observability, но observability предоставляет более полное понимание системы.



Зачем нужна Observability?

- Улучшение диагностики проблем
- Повышение надежности систем
- Ускорение процесса разработки и развертывания



АЛЕРТЫ

- Все что требует действий
- Обязательно разделение по важности



АЛЕРТЫ

- Все что требует действий
- Обязательно разделение по важности
- Все остальное решает визуализация и регламент ее обработки



ЛОГИ

Парсить

- Fluent[d|bit]
- Logstash
- …



ЛОГИ

Парсить

- Fluent[d|bit]
- Logstash
- …

Хранить

- ElasticSearch
- ClickHouse
- Loki
- …



ЛОГИ

Парсить	Хранить	Строить визуализацию
• Fluent[d|bit] • Logstash • …	• ElasticSearch • ClickHouse • Loki • …	• Grafana (можно интегрировать с метриками) • Kibana



ЛОГИ

Парсить	Хранить	Строить визуализацию	Алертить
• Fluent[d|bit] • Logstash • …	• ElasticSearch • ClickHouse • Loki • …	• Grafana (можно интегрировать с метриками) • Kibana	• Kibana • Grafana • Elastalert

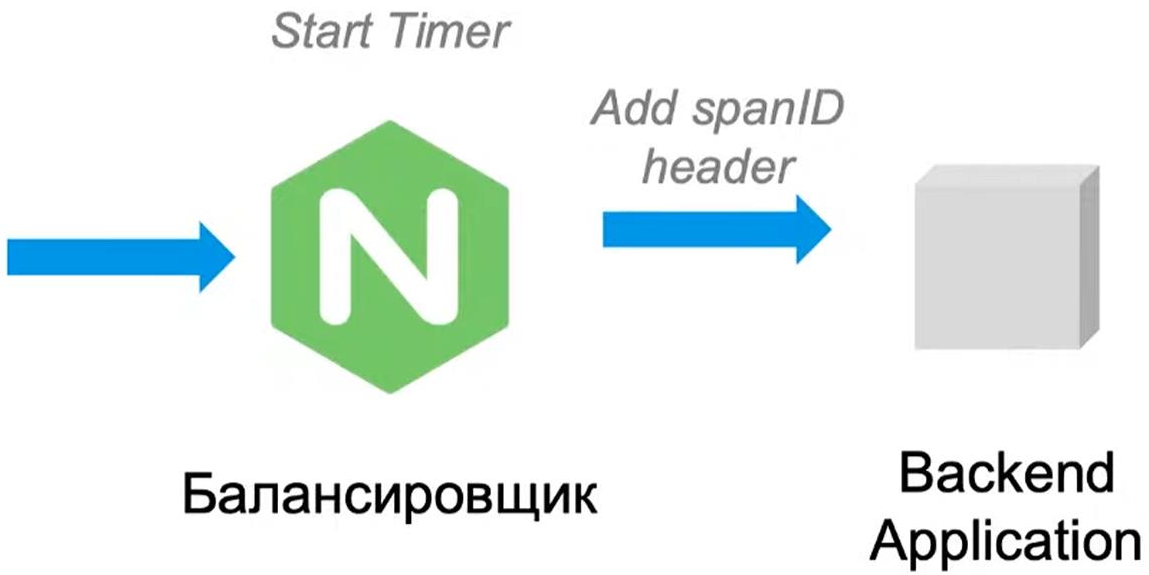


ТРЕЙСИНГ - ЗАЧЕМ

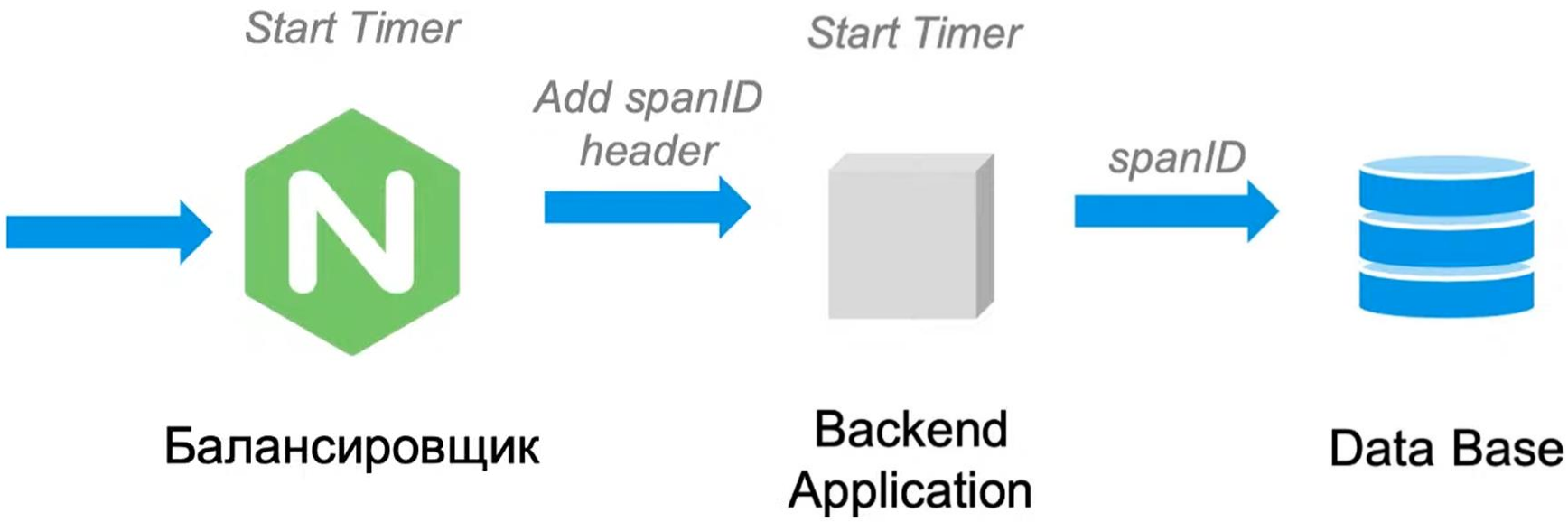
- Поиск узких мест
- Визуализация пути выполнения работы
- Сбор аналитической информации

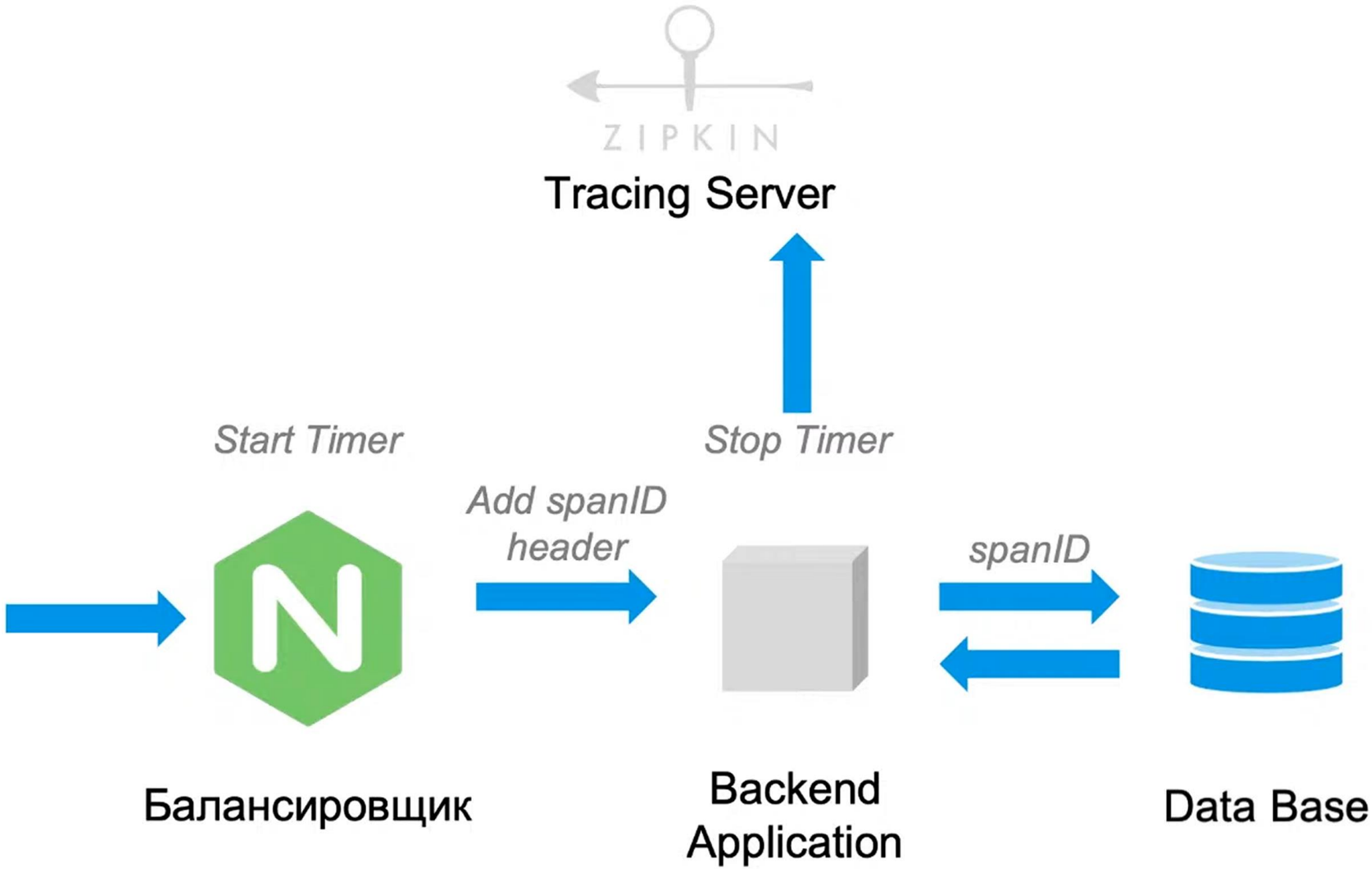


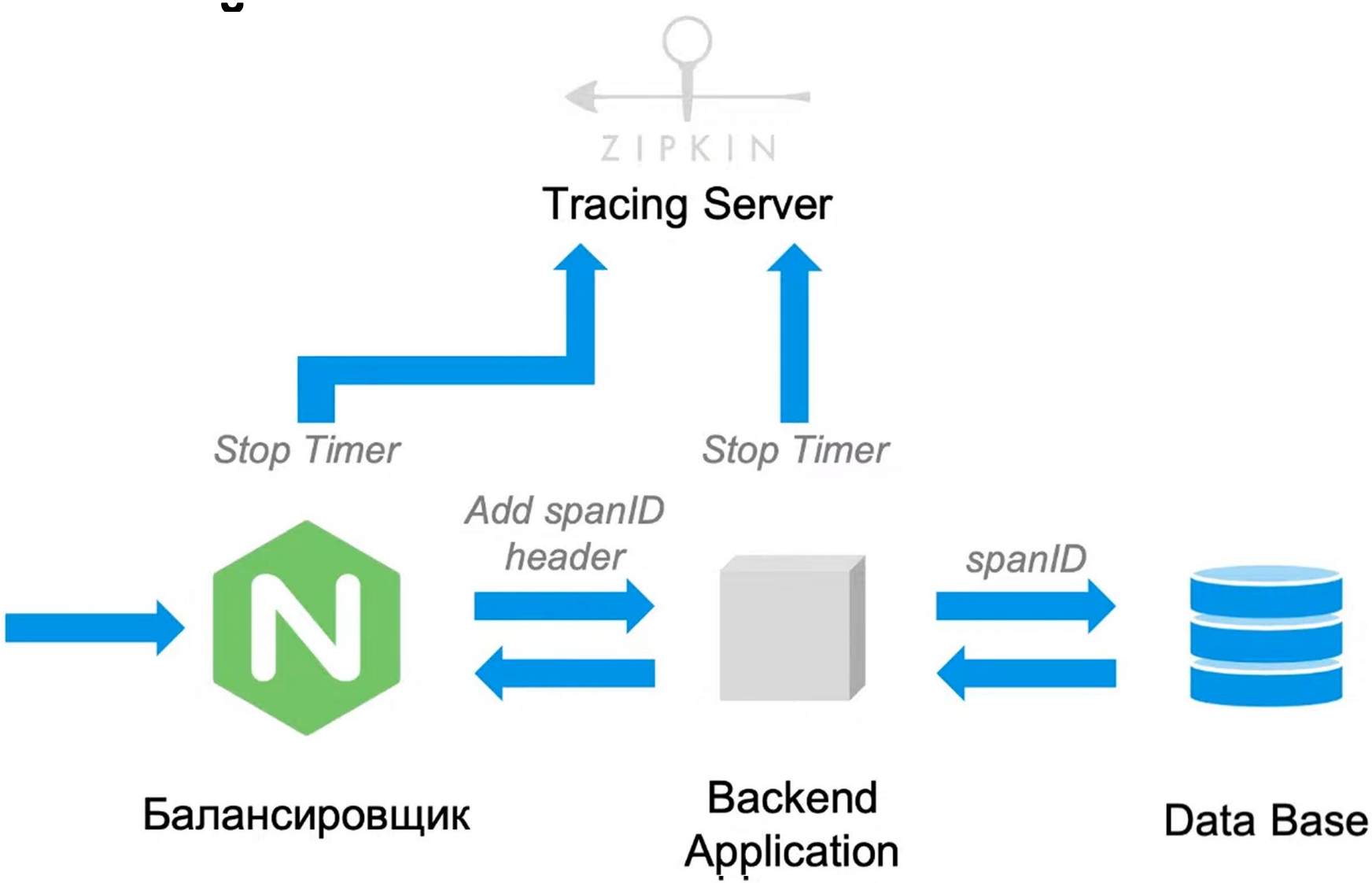
ТРЕЙСИНГ – КАК



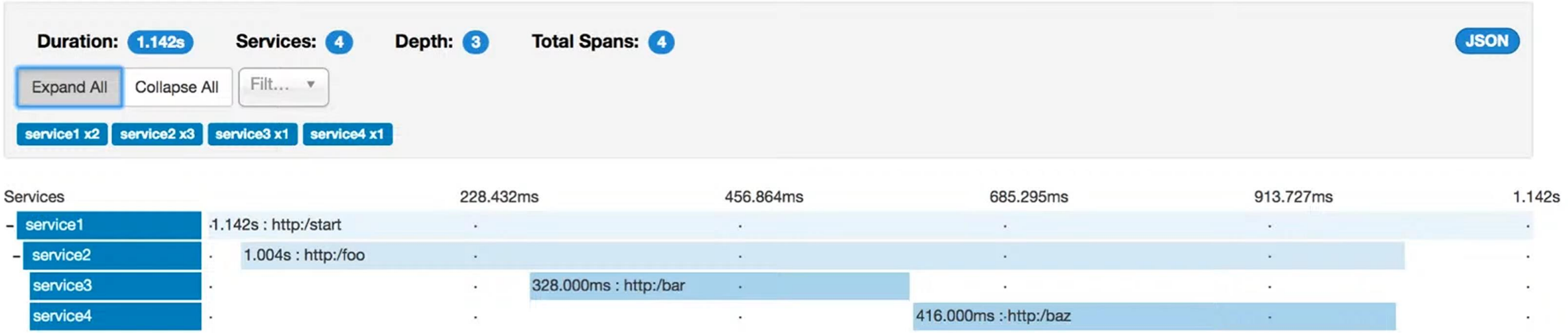
ТРЕЙСИНГ – КАК




ZIPKIN
Tracing Server
Start Timer
Stop Timer
Add spanID
header
spanID
Балансировщик
Backend
Application
Data Base


ZIPKIN
Tracing Server
Stop Timer
Stop Timer
Add spanID
header
spanID
Балансировщик
Backend
Application
Data Base

ТРЕЙСИНГ - ЧТО





ТРЕЙСИНГ - ЧЕМ







Application Performance Monitoring



Мониторинг + Логи +Трейсинг

Observability 2.0

1

Интеграция данных и автоматизация: Observability 2.0 акцентирует внимание на интеграции различных типов данных (метрики, логи, трассировки) в единую платформу, что позволяет автоматизировать процессы мониторинга и анализа, улучшая скорость реакции на инциденты и повышая предсказуемость систем.

2

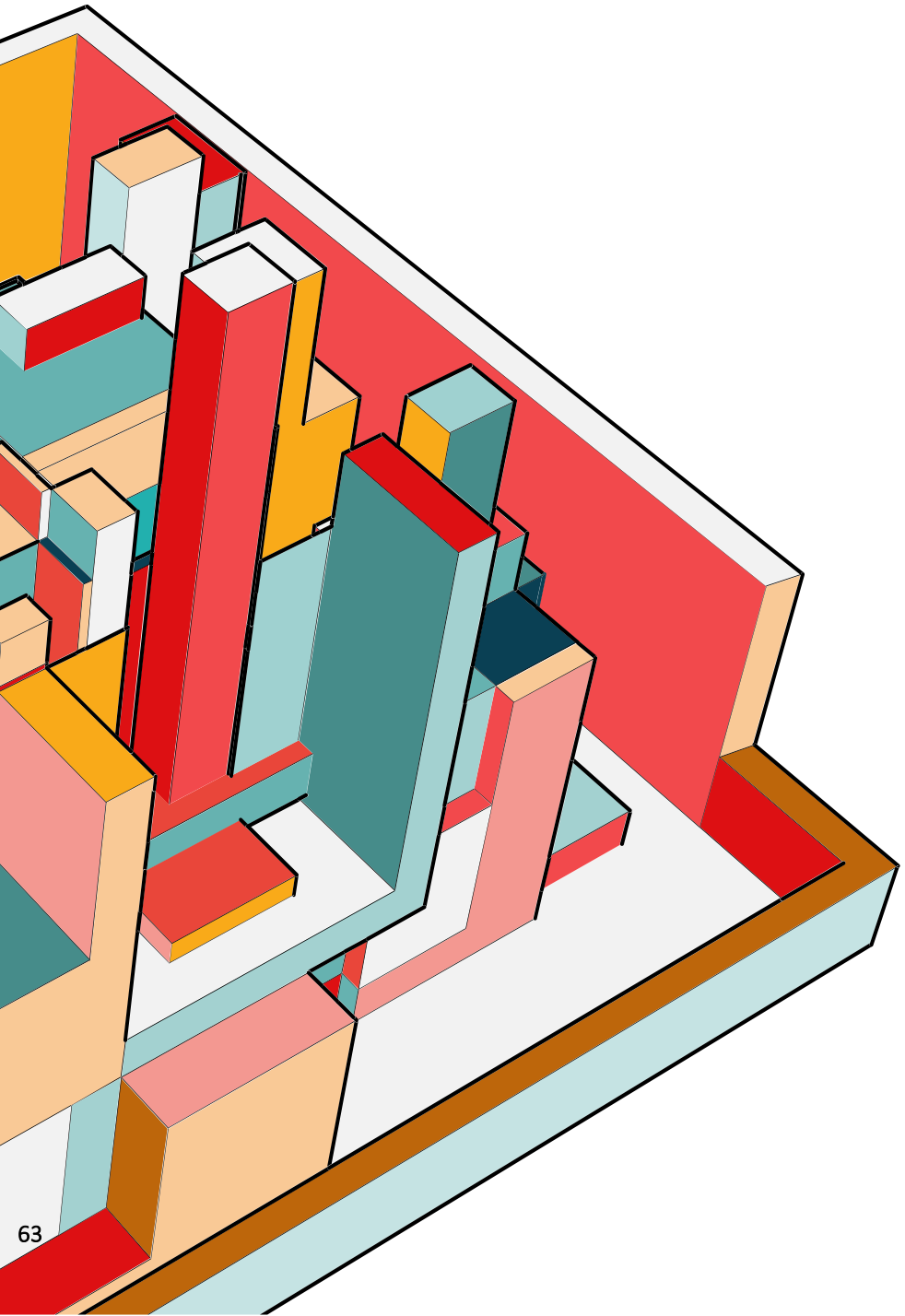
Контекстуальный анализ и AI/ML: В Observability 2.0 активно используются технологии искусственного интеллекта и машинного обучения для контекстуального анализа данных, что позволяет выявлять аномалии и предсказывать проблемы до их возникновения, обеспечивая проактивное управление производительностью и надежностью систем.



Тренды в Observability

- Увеличение значимости автоматизации
- Развитие AI/ML для анализа данных
- Переход к проактивному мониторингу

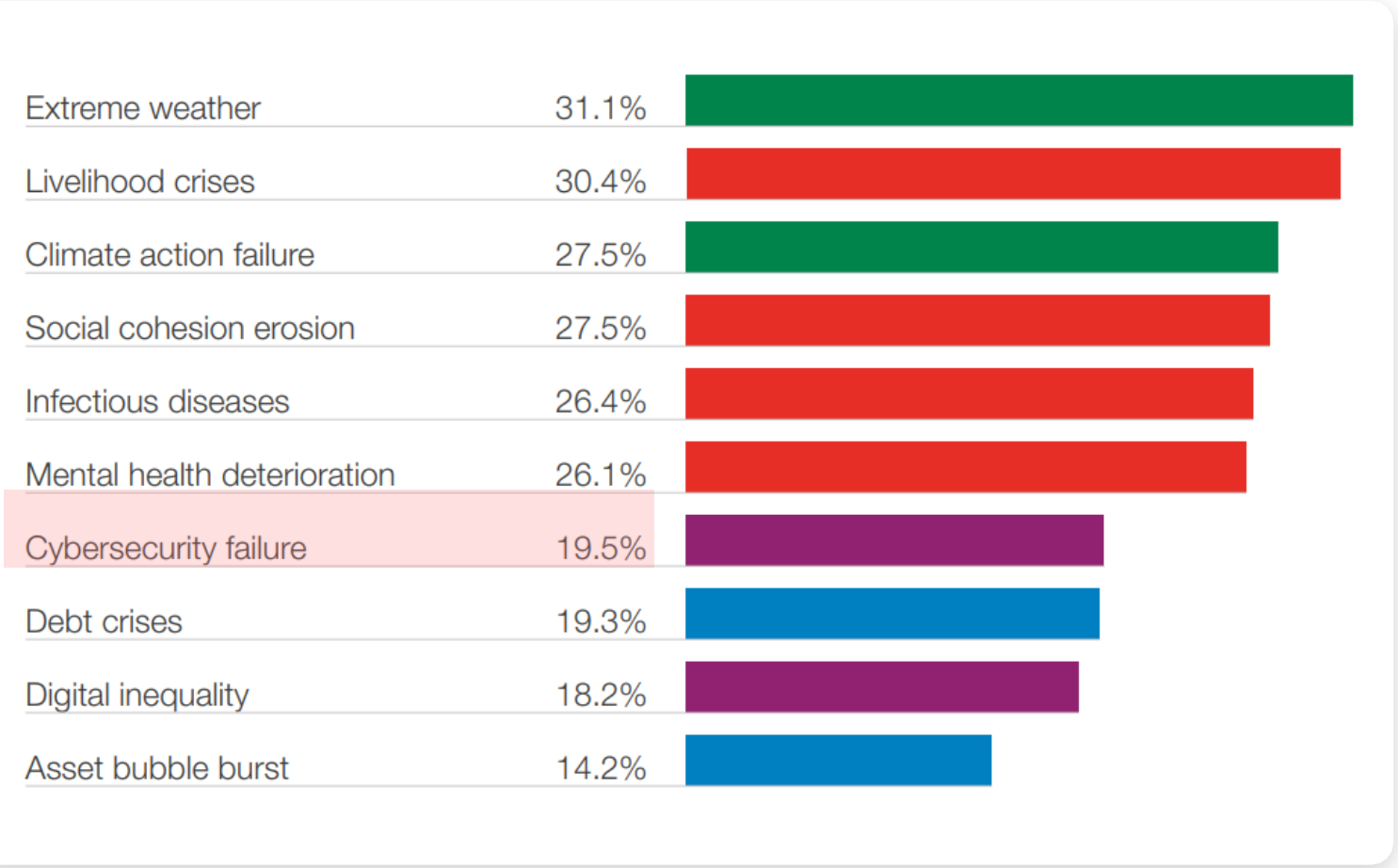




ПРЕДПОСЫЛКИ DEVSECOPS

В МИРЕ.
ГЛОБАЛЬНЫЕ РИСКИ 2022-2024

Риски, которые являются мировыми критическими угрозами



-World Economic Forum. The Global Risks Report 2022

3 миллиона

мировой дефицит специалистов по кибербезопасности

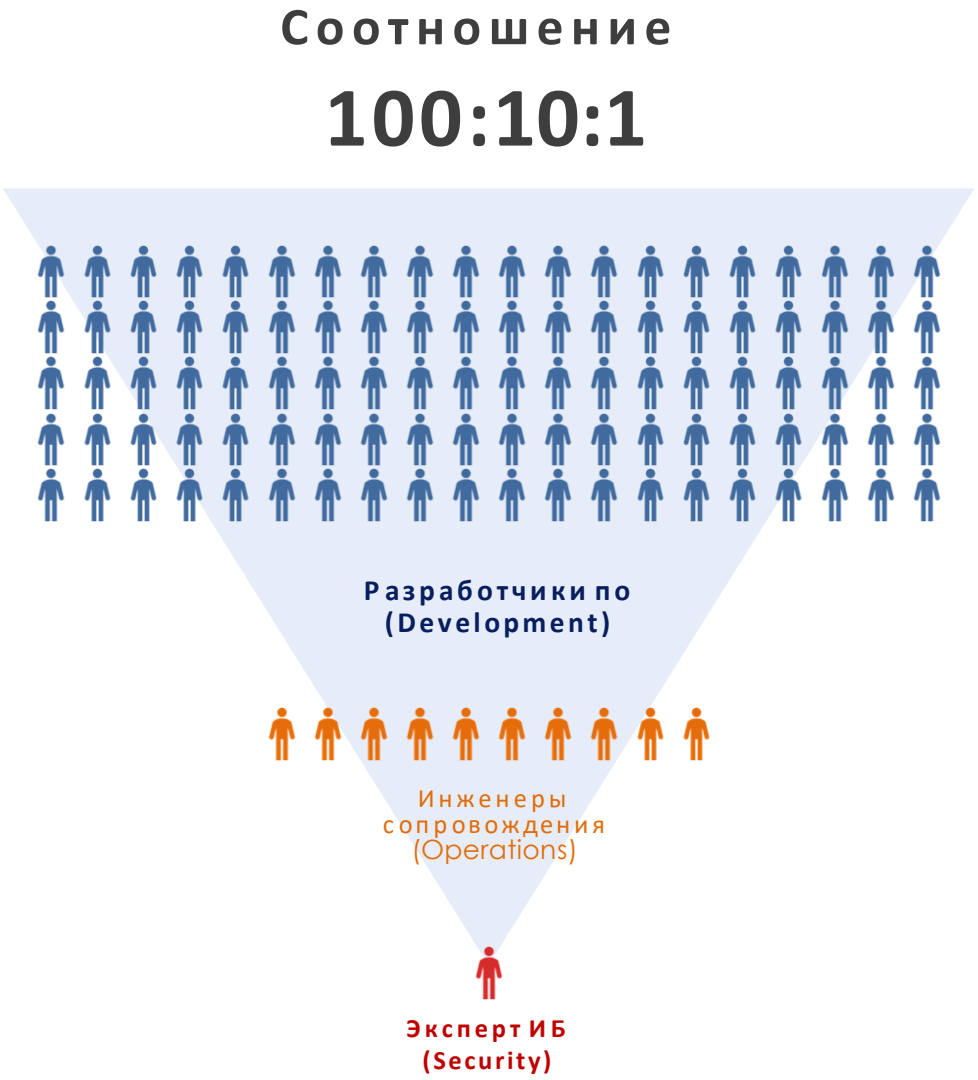
800 миллиардов долларов

предполагаемый рост объема рынка цифровых услуг к 2024 году

95%

проблем в области кибербезопасности вызваны человеческим фактором

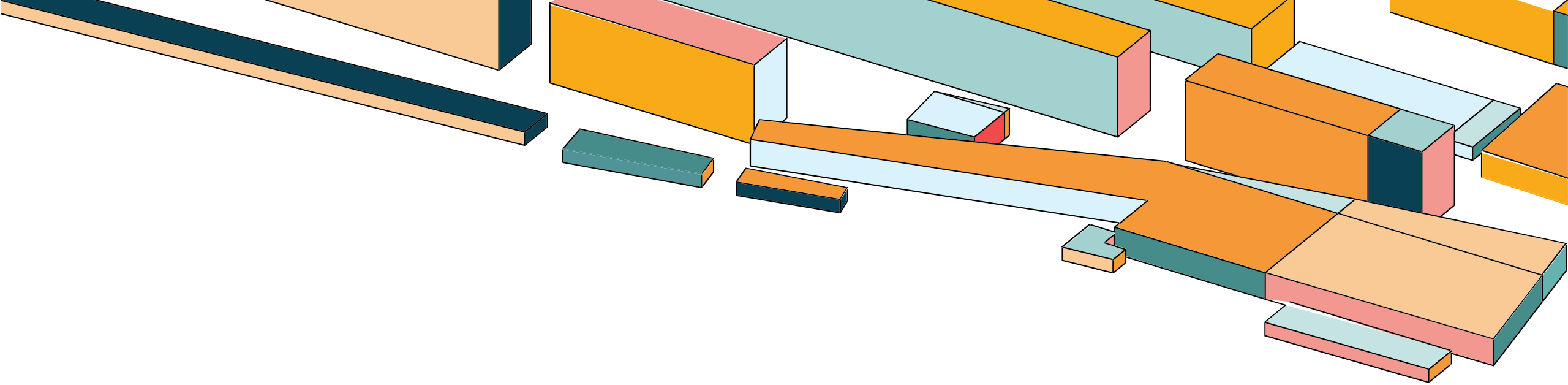
Ускорение разработки ПО создает вызовы



90% инцидентов безопасности вызваны дефектами исходного кода

21% утечек данных вызван уязвимостями программного обеспечения

1 из 3 приложений содержит критическую уязвимость

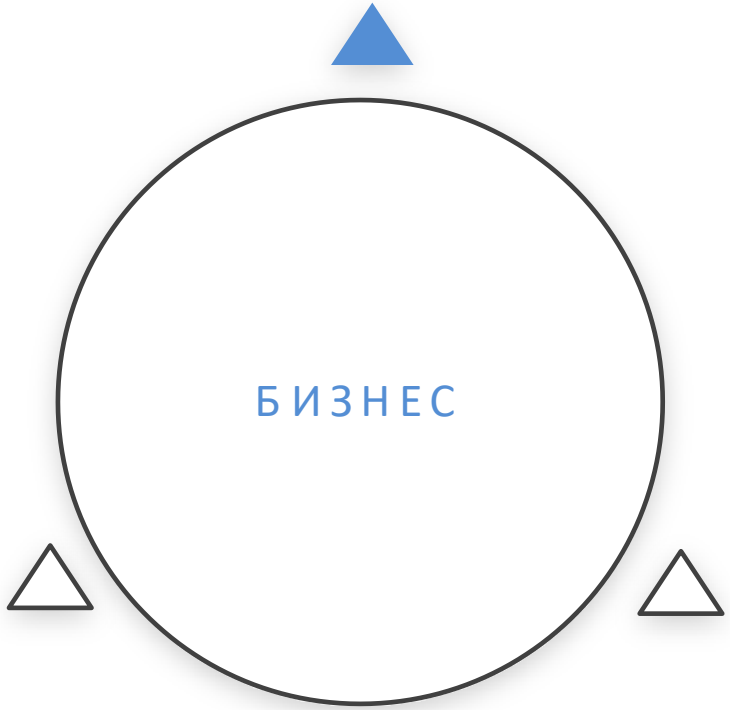


ИНДУСТРИАЛЬНЫЕ ФАКТОРЫ

ИНДУСТРИАЛЬНЫЕ ФАКТОРЫ

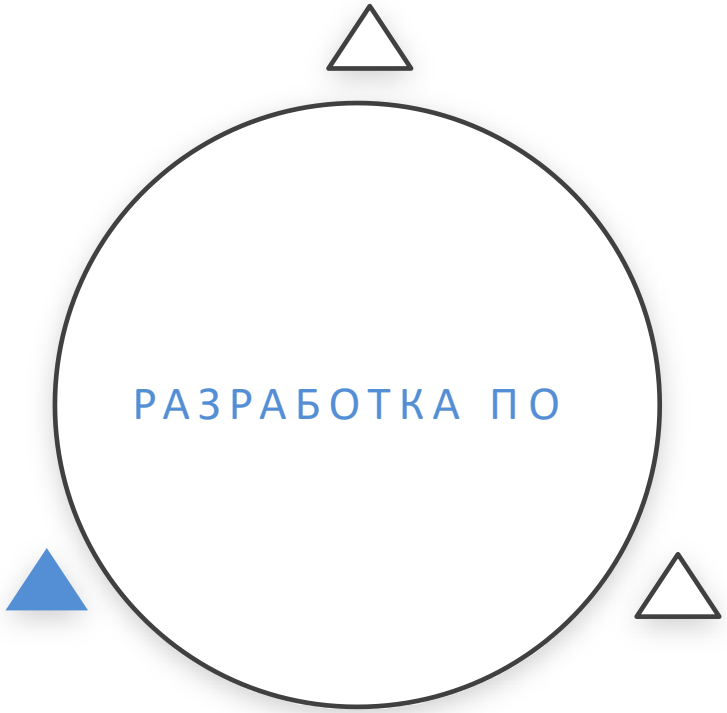


ИНДУСТРИАЛЬНЫЕ ФАКТОРЫ



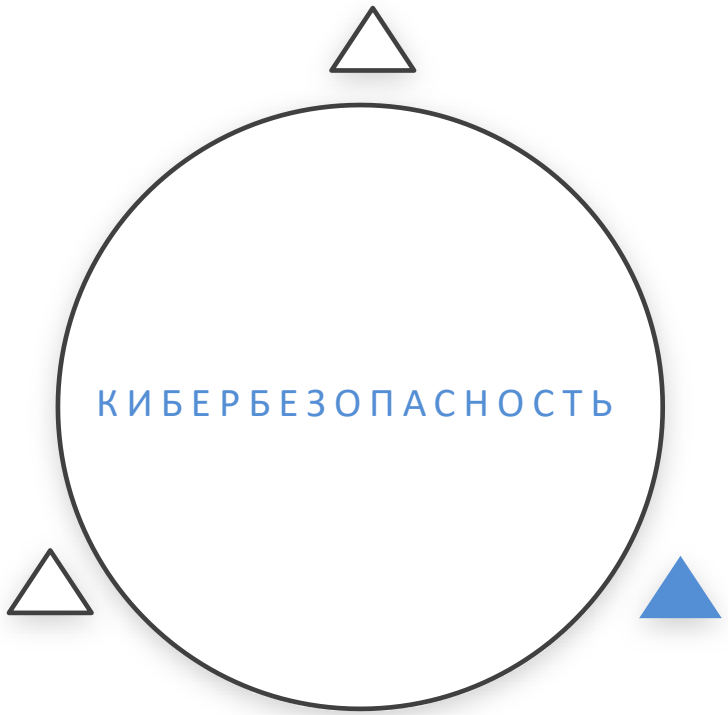
- Непрерывный рост количества программных продуктов
- Рост количества вовлеченных сторон в компании
- Time-to-Market: непрерывность и безопасность становятся ключевыми критериями успеха в бизнесе

ИНДУСТРИАЛЬНЫЕ ФАКТОРЫ

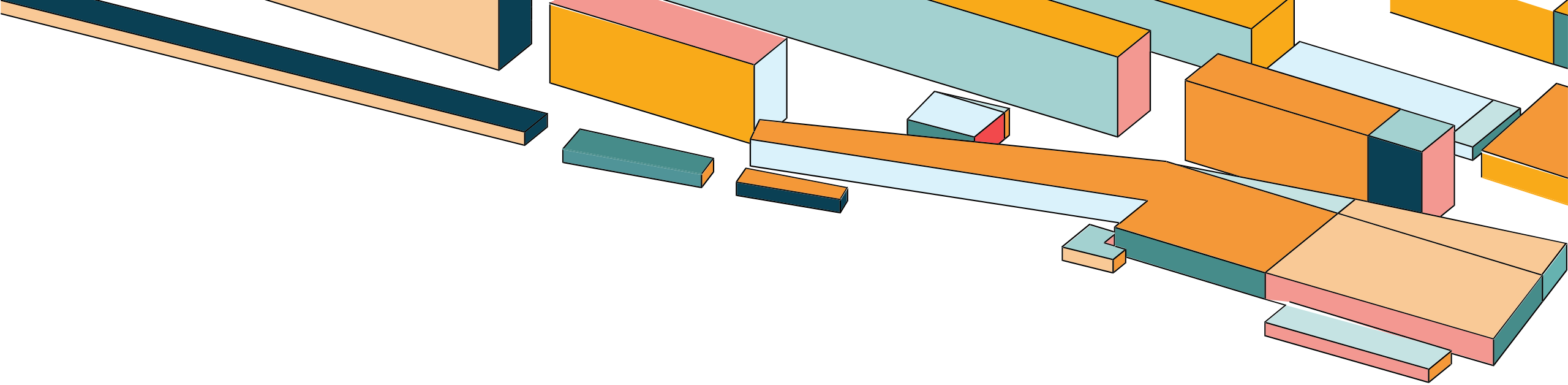


- Сокращение циклов разработки ПО
- Увеличение количества релизов
- Переход в микросервисную архитектуры приложений
- Появление распределенных команд разработки

ИНДУСТРИАЛЬНЫЕ ФАКТОРЫ

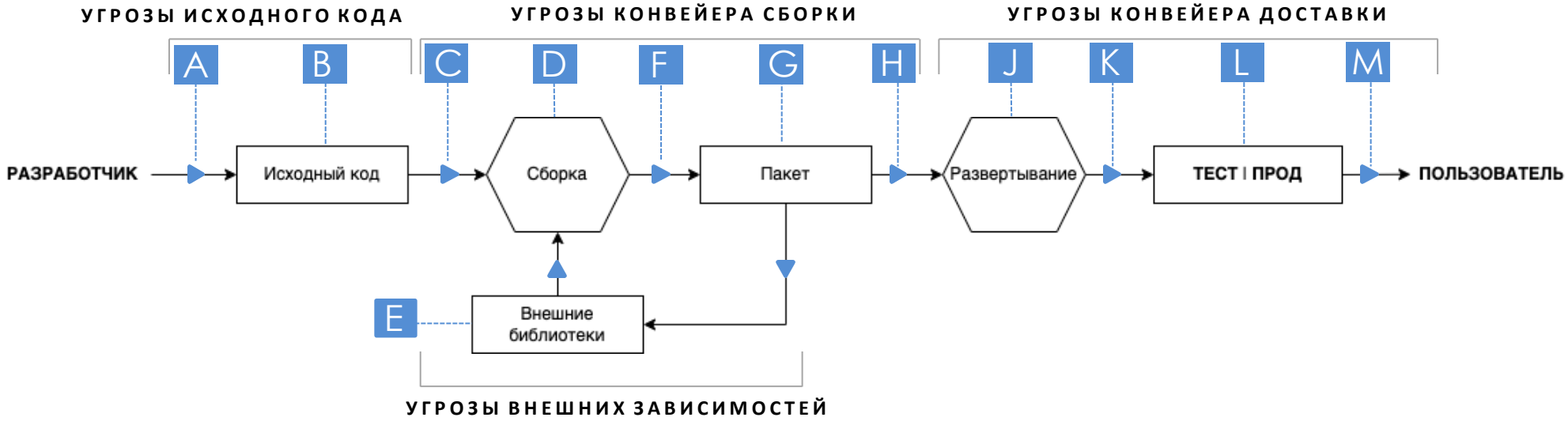


- Рост технического долга дефектов ИБ
- Большое количество ложных уязвимостей (False-Positives)
- Нехватка квалифицированных кадров
- Невозможность управления средствами ИБ вручную



ЦИФРОВЫЕ УГРОЗЫ ЭВОЛЮЦИОНИРУЮТ

ЦИФРОВЫЕ УГРОЗЫ ПОСТОЯННО ЭВОЛЮЦИОНИРУЮТ



УГРОЗЫ ИСХОДНОГО КОДА

A Обход процессаревью кода

B Компрометация системы управления исходным кодом

УГРОЗЫ КОНВЕЙЕРА СБОРКИ

C Несанкционированная модификация кода

D Компрометация системы сборки

F Обход CI/CD-процесса

G Компрометация репозитория

H Использованиеуязвимого пакета

УГРОЗЫ ВНЕШНИХ ЗАВИСИМОСТЕЙ

E Использованиеуязвимых библиотек / зловредных компонент с открытым исходным кодом

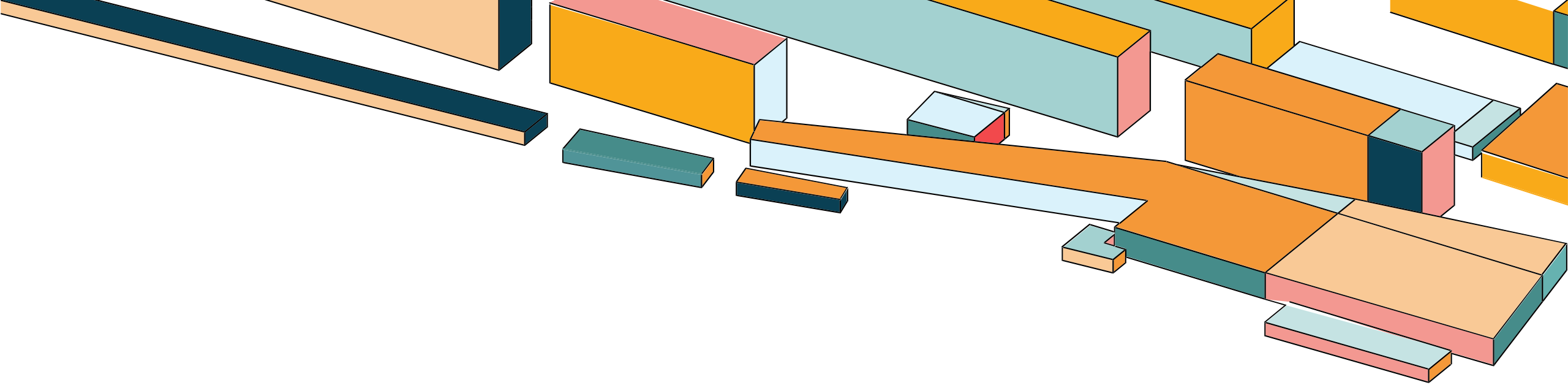
УГРОЗЫ КОНВЕЙЕРА ДОСТАВКИ

J Компрометация системы развертывания

K Использованиеуязвимых контейнеров

L Компрометация инфраструктурного окружения

M Эксплуатация уязвимых интерфейсов



ЦЕЛИ DEVSECOPS



Time-to-Market

Скорость реализации новых бизнес-требований

- Снизить время вывода на рынок
- Увеличивать объемы выпуска бизнес-функций
- Сохранять качество и защищенность ПО



COMPLIANCE

- Обеспечить контроль соответствия индустриальным стандартам
- Обеспечить соответствие ПО регуляторным требованиям
- Обеспечить лицензионную чистоту используемых OpenSource-компонентов



DIGITAL BUSINESS CONTINUITY & SECURITY

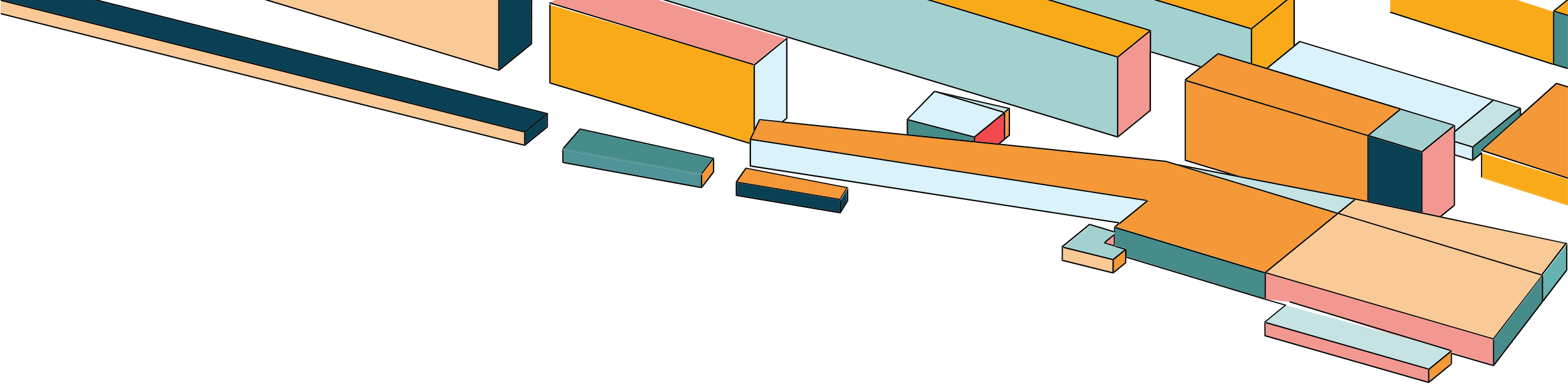
- Повысить киберустойчивость программных продуктов
- Снизить плотность дефектов в коде
- Повысить скорость устранения технического долга ИБ

Цели DevSecOps



DEVELOPMENT MATURITY

- Повысить продуктивность разработчиков
- Интегрировать непрерывный анализ защищенности в конвейер разработки ПО
- Повысить уровень экспертизы разработчиков в ИБ



МЕТОДОЛОГИЯ DEVSECOPS

DEVSECOPS ЗАТРАГИВАЕТ ВСЕ ОБЛАСТИ РАЗРАБОТКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

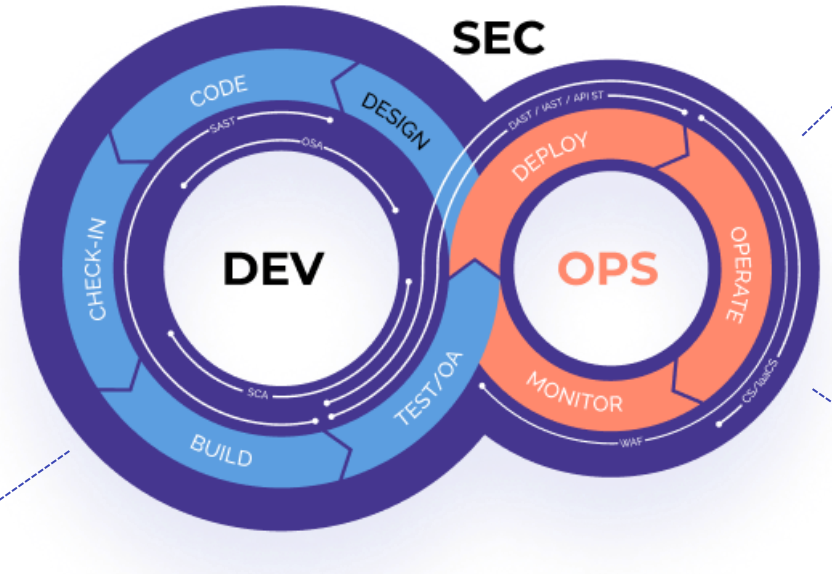


ПОСТРОЕНИЕ ПЛАТФОРМЫ РАЗРАБОТКИ ЗАЩИЩЕННОГО ПО

Анализ библиотек с открытым исходным кодом
Open-Source Analysis

Статистический анализ кода
SAST Static Application Security Testing

Динамический анализ
DAST Dynamic Application Security Testing



Компонентный анализ ПО
SCA Software Composition Analysis

Анализ защищенности контейнеров
CS Container Security

Оркестрация и корреляция
ASOC Application Security Orchestration & Correlation

СПАСИБО!

Виденин Сергей

@videninserg

